

13 Filtering data

This chapter covers

- Cascading state transitions
- Listening to and intercepting events
- Auditing and versioning with Hibernate Envers
- Filtering data dynamically

In this chapter, we'll analyze many different strategies for *filtering* data as it passes through the Hibernate engine. When Hibernate loads data from the database, we can transparently restrict the data seen by the application with a filter. When Hibernate stores data in the database, we can listen to the event and execute secondary routines: for example, we could write an audit log or assign a tenant identifier to a record.

In the four main sections of this chapter, we'll explore the following data-filtering features and APIs:

- First you'll learn to react to state changes of an entity instance and *cascade the state change* to associated entities. For example, when a `User` is saved, Hibernate can transitively and automatically save all related `BillingDetails`. When an `Item` is deleted, Hibernate can delete all `Bid` instances associated with that `Item`. We can enable this standard JPA feature with special attributes in the entity association and collection mappings.
- The Jakarta Persistence standard includes lifecycle *callbacks* and *event listeners*. An event listener is a class we write with special methods, called by Hibernate when an entity instance changes state, such as after Hibernate loads it or is about to delete it from the database. These callback methods can also be on the entity classes and marked with special annotations. This gives us an opportunity to execute custom side effects when a transition occurs. Hibernate also has several proprietary extension points that allow interception of lifecycle events at a lower level within its engine.
- A common side effect is writing an *audit log*; such a log typically contains information about the data that was changed, when the change was made, and who made the modification. A more sophisticated auditing system might require storing several versions of data and *temporal views*; for example, we might want to ask Hibernate to load data "as it was last week." This being a complex problem, we'll introduce Hibernate Envers, a subproject dedicated to versioning and auditing in JPA applications.

- Finally, we'll examine *data filters*, which are also available as a proprietary Hibernate API. These filters add custom restrictions to SQL `SELECT` statements executed by Hibernate. Hence, we can effectively define a custom limited view of the data in the application tier. For example, we could apply a filter that restricts loaded data by sales region or any other authorization criteria.

We'll start with cascading options for transitive state changes.

NOTE To be able to execute the examples from the source code, you'll first need to run the `Ch13.sql` script.

13.1 Cascading state transitions

When an entity instance changes state—such as when goes from being *transient* to *persistent*—associated entity instances may also be included in this state transition. This *cascading* of state transitions isn't enabled by default; each entity instance has an independent lifecycle. But for some associations between entities, we may want to implement fine-grained lifecycle dependencies.

For example, in section 8.3 we created an association between the `Item` and `Bid` entity classes. In this case, not only did we make the bids of an `Item` automatically persistent when they were added to an `Item`, but they were also automatically deleted when the owning `Item` was deleted. We effectively made `Bid` an entity class that was dependent on another entity, `Item`.

The cascading settings we enabled in that association mapping were `CascadeType.PERSIST` and `CascadeType.REMOVE`. We also investigated the special `orphanRemoval` switch and how cascading deletion at the database level (with the foreign key `ON DELETE` option) affects the application.

So we briefly pointed out how we could work with *cascading* state in chapter 8. In this section, we'll analyze some other, rarely used, cascading options.

13.1.1 Available cascading options

Table 13.1 summarizes the most important cascading options available in Hibernate. Note how each is linked with an `EntityManager` or `Session` operation.

Table 13.1 Cascading options for entity association mappings

Option	Description
<code>CascadeType.PERSIST</code>	When an entity instance is stored with <code>EntityManager #persist()</code> , at flush time any associated entity instances are also made persistent.
<code>CascadeType.REMOVE</code>	When an entity instance is deleted with <code>EntityManager #remove()</code> , at flush time any associated entity instances are also removed.
<code>CascadeType.DETACH</code>	When an entity instance is evicted from the persistence context with <code>EntityManager#detach()</code> , any associated entity instances are also detached.
<code>CascadeType.MERGE</code>	When a transient or detached entity instance is merged into a persistence context with <code>EntityManager#merge()</code> , any associated transient or detached entity instances are also merged.
<code>CascadeType.REFRESH</code>	When a persistent entity instance is refreshed with <code>EntityManager#refresh()</code> , any associated persistent entity instances are also refreshed.
<code>CascadeType.REPLICATE</code>	When a detached entity instance is copied into a database with <code>Session#replicate()</code> , any associated detached entity instances are also copied.
<code>CascadeType.ALL</code>	This is shorthand to enable all cascading options for the mapped association.

There are more cascading options defined in the `org.hibernate.annotations.CascadeType` enumeration. Today, though, the only interesting option is `REPLICATE` and the `Session#replicate()` operation. All other `Session` operations have a standardized equivalent or an alternative on the `EntityManager` API, so we can ignore these settings.

We've already examined the `PERSIST` and `REMOVE` options. Let's analyze transitive detachment, merging, refreshing, and replication.

13.1.2 Transitive detachment and merging

We want to retrieve an `Item` and its `bids` from the database and work with this data in detached state. The `Bid` class maps this association with a `@ManyToOne`. It's bidirectional with this `@OneToMany` collection mapping in `Item`:

```
Path: Ch13/cascade/src/main/java/com/manning/javapersistence/ch13/filtering
➡ /cascade/Item.java
```

```
\1
public class Item {
    @OneToMany(
        mappedBy = "item",
        cascade = {CascadeType.DETACH, CascadeType.MERGE}
    )
    private Set<Bid> bids = new HashSet<>();
    // . . .
}
```

Transitive detachment and merging is enabled with the `DETACH` and `MERGE` cascade types. Now we can load the `Item` and initialize its `bids` collection:

```
Path: Ch13/cascade/src/test/java/com/manning/javapersistence/ch13/filtering
➡ /Cascade.java
```

```
Item item = em.find(Item.class, ITEM_ID);
assertEquals(2, item.getBids().size());    ①
em.detach(item);                          ②
```

① Accessing `item.getBids()` initializes the `bids` collection (it is lazily initialized).

② The `EntityManager#detach()` operation is cascaded: it evicts the `Item` instance from the persistence context as well as all `bids` in the collection. If the `bids` aren't loaded, they aren't detached. (Of course, we could have closed the persistence context, effectively detaching *all* loaded entity instances.)

In detached state, we can change the `Item#name`, create a new `Bid`, and link it with the `Item`:

```
Path: Ch13/cascade/src/test/java/com/manning/javapersistence/ch13/filtering
➡ /Cascade.java
```

```
item.setName("New Name");
Bid bid = new Bid(new BigDecimal("101.00"), item);
item.addBid(bid);
```

Because we're working with detached entity state and collections, we have to pay extra attention to identity and equality. As discussed in sec-

tion 10.3, we should override the `equals()` and `hashCode()` methods on the `Bid` entity class:

```
Path: Ch13/cascade/src/main/java/com/manning/javapersistence/ch13/filtering
➡ /cascade/Bid.java

@Entity
public class Bid {
    @Override
    public boolean equals(Object o) {
        if (this == o) return true;
        if (o instanceof Bid bid) {
            return Objects.equals(id, bid.id) &&
                Objects.equals(amount, bid.amount) &&
                Objects.equals(item, bid.item);
        }
        return false;
    }

    @Override
    public int hashCode() {
        return Objects.hash(id, amount, item);
    }
}
```

Two `Bid` instances are *equal* when they have the same `id`, the same `amount`, and are linked with the same `Item`.

Once we're done with the modifications in detached state, the next step is to store the changes. Using a new persistence context, we can merge the detached `Item` and let Hibernate detect the changes.

Using the `merge` method, Hibernate will merge a detached instance. First it checks whether the persistence context already contains an entity with the given identifier value. If there isn't any, the entity is loaded from the database. Hibernate is smart enough to know that it will also need the referenced entities during merging, so it fetches them right away in the same SQL query.

When flushing the persistence context, Hibernate detects whether some property of the entity changed during merging. Referenced entities may also be stored:

```
Path: Ch13/cascade/src/test/java/com/manning/javapersistence/ch13/filtering
➡ /Cascade.java

Item mergedItem = em.merge(item);
// select i.*, b.*
// from ITEM i
// left outer join BID b on i.ID = b.ITEM_ID
// where i.ID = ?
for (Bid b : mergedItem.getBids()) {
    assertNotNull(b.getId());
}
em.flush();
```

```
// update ITEM set NAME = ? where ID = ?
// insert into BID values (?, ?, ?, . . . )
```

Ⓐ Hibernate merges the detached `item`. There isn't any `Item` with the given identifier value, so the `Item` is loaded from the database.

Hibernate fetches the `bids` during merging in the same SQL query.

Hibernate then copies the detached `item` values into the loaded instance, which it returns to us in persistent state. The same procedure is applied to every `Bid`, and Hibernate will detect that one of the `bids` is new.

Ⓑ Hibernate made the new `Bid` persistent during merging. It now has an identifier value assigned.

Ⓒ When we flush the persistence context, Hibernate detects that the name of the `Item` changed during merging. The new `Bid` will also be stored.

Cascaded merging with collections is a powerful feature; consider how much code we would have to write without Hibernate to implement this functionality.

Eagerly fetching associations when merging

In the final code example in section 13.1.2, we said that Hibernate is smart enough to load the `Item#bids` collection when we merge a detached `Item`. Hibernate always loads entity associations eagerly with a `JOIN` when merging if `CascadeType.MERGE` is enabled for the association. This is smart in the aforementioned case, where the `Item#bids` were initialized, detached, and modified. It's necessary and optimal for Hibernate to load the collection when merging with a `JOIN`, but if we merge an `Item` instance with an uninitialized `bids` collection or an uninitialized `seller` proxy, Hibernate will fetch the collection and proxy with a `JOIN` when merging. The merge initializes these associations on the managed `Item` it returns. `CascadeType.MERGE` causes Hibernate to ignore and effectively override any `FetchType.LAZY` mapping (as allowed by the JPA specification).

Our next example is less sophisticated, enabling cascaded refreshing of related entities.

13.1.3 Cascading refresh

The `User` entity class has a one-to-many relationship with `BillingDetails`: each user of the application may have several credit cards, bank accounts, and so on. If you would like to review the `BillingDetails` class, take a look at the mappings in chapter 7.

We can map the relationship between `User` and `BillingDetails` as a unidirectional one-to-many entity association:

```
Path: Ch13/cascade/src/main/java/com/manning/javapersistence/ch13/filtering
➡ /cascade/User.java
```

```
@Entity
@Table(name = "USERS")
public class User {
    @OneToMany(cascade = {CascadeType.PERSIST, CascadeType.REFRESH})
    @JoinColumn(name = "USER_ID", nullable = false)
    private Set<BillingDetails> billingDetails = new HashSet<>();
    // . . .
}
```

The cascading options enabled for this association are `PERSIST` and `REFRESH`. The `PERSIST` option simplifies storing billing details; they become persistent when we add an instance of `BillingDetails` to the collection of an already persistent `User`.

The `REFRESH` cascading option ensures that when we reload the state of a `User` instance, Hibernate will also refresh the state of each `BillingDetails` instance linked to the `User`. For example, when we `refresh()` the managed `User` instance, Hibernate cascades the operation to the managed `BillingDetails` and refreshes each with an SQL `SELECT`. If none of these instances remain in the database, Hibernate throws an `EntityNotFoundException`. Then, Hibernate refreshes the `User` instance and eagerly loads the entire `billingDetails` collection to discover any new `BillingDetails`:

```
Path: Ch13/cascade/src/test/java/com/manning/javapersistence/ch13/filtering
➡ /Cascade.java
```

```
User user = em.find(User.class, USER_ID);                                ❸
assertEquals(2, user.getBillingDetails().size());                        ❹
for (BillingDetails bd : user.getBillingDetails()) {
    assertEquals("John Doe", bd.getOwner());
}
// Someone modifies the billing information in the database!
em.refresh(user);                                                         ❺
// select * from CREDITCARD join BILLINGDETAILS where ID = ?
// select * from BANKACCOUNT join BILLINGDETAILS where ID = ?
// select * from USERS
// left outer join BILLINGDETAILS
// left outer join CREDITCARD
// left outer JOIN BANKACCOUNT
// where ID = ?
for (BillingDetails bd : user.getBillingDetails()) {
    assertEquals("Doe John", bd.getOwner());
}
```

❸ An instance of `User` is loaded from the database.

❹ Its lazy `billingDetails` collection is initialized when we iterate through the elements or when we call `size()`.

© When we `refresh()` the managed `User` instance, Hibernate cascades the operation to the managed `BillingDetails` and refreshes each with an SQL `SELECT`.

This is a case where Hibernate isn't as smart as it could be. First, it executes an SQL `SELECT` for each `BillingDetails` instance that's in the persistence context and referenced by the collection. Then it loads the entire collection again to find any added `BillingDetails`. Hibernate could obviously do this with one `SELECT`.

We would like to refresh the record after it's modified by another transaction, so we have to keep in mind that the default transaction isolation level for MySQL is `REPEATABLE_READ`, while for most other databases it is `READ_COMMITTED`. We started one transaction and then a second, which committed its changes before the first one performed the refresh operation. For the first transaction to be able to see the changes from the second, we need to change the isolation level on the JDBC driver. That is why we provided the following configuration URL:

```
Path: Ch13/cascade/src/main/resources/META-INF/persistence.xml
```

```
<property name="javax.persistence.jdbc.url"
value="jdbc:mysql://localhost:3306/CH13_CASCADE?
➤ sessionVariables=transaction_isolation=
➤ 'READ-COMMITTED' &serverTimezone=UTC"/>
```

As the change is made only at the configuration level, the code will continue to work correctly using different databases, as long as the `persistence.xml` file contains the correct configuration.

The last cascading option is for the Hibernate-only `replicate()` operation.

13.1.4 Cascading replication

We first examined replication in section 10.2.7. This nonstandard operation is available on the Hibernate `Session` API. Its main use case is copying data from one database into another.

Consider this many-to-one entity association mapping between `Item` and `User`:

```
Path: Ch13/cascade/src/main/java/com/manning/javapersistence/ch13/filtering
➤ /cascade/Item.java
```

```
@Entity
public class Item {
    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "SELLER_ID", nullable = false)
    @org.hibernate.annotations.Cascade(
        org.hibernate.annotations.CascadeType.REPLICATE
    )
    private User seller;
```

```

    // . . .
}

```

Here we enable the `REPLICATE` cascading option with a Hibernate annotation. Next, we'll load an `Item` and its `seller` from the source database:

```

Path: Ch13/cascade/src/test/java/com/manning/javapersistence/ch13/filtering
➡ /Cascade.java

```

```

em = emf.createEntityManager();
em.getTransaction().begin();
Item item = em.find(Item.class, ITEM_ID);
assertNotNull(item.getSeller().getUsername()); ①
em.getTransaction().commit();
em.close();

```

① Initialize `Item#seller` lazily.

After we close the persistence context, the `Item` and the `User` entity instances are in detached state. Next, we connect to the database and write the detached data:

```

Path: Ch13/cascade/src/test/java/com/manning/javapersistence/ch13/filtering
➡ /Cascade.java

```

```

EntityManager otherDatabase = // . . . get EntityManager
otherDatabase.getTransaction().begin();
otherDatabase.unwrap(Session.class)
    .replicate(item, ReplicationMode.OVERWRITE);
// select ID from ITEM where ID = ?
// select ID from USERS where ID = ?
otherDatabase.getTransaction().commit();
// update ITEM set NAME = ?, SELLER_ID = ?, . . . where ID = ?
// update USERS set USERNAME = ?, . . . where ID = ?
otherDatabase.close();

```

When we call `replicate()` on the detached `Item`, Hibernate executes SQL `SELECT` statements to find out whether the `Item` and its `seller` are already present in the database. Then, on commit, when the persistence context is flushed, Hibernate writes the values of the `Item` and the `seller` into the target database. In the previous example, these rows were already present, so we'd see an `UPDATE` of each, overwriting the values in the database. If the target database doesn't contain the `Item` or `User`, two `INSERT`s are made.

13.2 Listening to and intercepting events

In this section, we'll analyze three different APIs for custom event listeners and persistence lifecycle interceptors available in JPA and Hibernate. They allow us to do several things:

- Use the standard JPA lifecycle callback methods and event listeners.
- Write a proprietary `org.hibernate.Interceptor` and activate it on a `Session`.
- Use extension points of the Hibernate core engine with the `org.hibernate.event` service provider interface (SPI).

Let's start with the standard JPA callbacks. They offer easy access to persist, load, and remove lifecycle events.

13.2.1 JPA event listeners and callbacks

Let's say we want to log a message whenever a new entity instance is stored. An entity listener class must have an implicit or explicit public no-argument constructor. It doesn't have to implement any special interfaces. An entity listener is stateless; the JPA engine automatically creates and destroys it. This means it can be difficult to get more contextual information when we need it, but we'll demonstrate some possibilities.

First we'll write a lifecycle event listener with a callback method, annotated with `@PostPersist`, as shown in the following listing. We can annotate any method of an entity listener class as a callback method for persistence lifecycle events.

Listing 13.1 Notifying an admin when an entity instance was stored

Path: Ch13/callback/src/main/java/com/manning/javapersistence/ch13
 ➡ /filtering/callback/PersistEntityListener.java

```
public class PersistEntityListener {
    @PostPersist
    public void logMessage(Object entityInstance) {
        User currentUser = CurrentUser.INSTANCE.get();
        Log log = Log.INSTANCE;
        log.save(
            "Entity instance persisted by "
            + currentUser.getUsername()
            + ": "
            + entityInstance
        );
    }
}
```

Ⓐ The `logMessage()` method, annotated with `@PostPersist`, is invoked after a new entity instance is stored in the database.

Ⓑ We want the contextual information of the currently logged-in user and access to log information. A primitive solution is to use thread-local variables and singletons; the source for `CurrentUser` and `Log` is in the example code.

A callback method of an entity listener class has a single `Object` parameter: the entity instance involved in the state change. If we only enable the callback for a particular entity type, we may declare the argument as

that specific type. The callback method can have any kind of access; it doesn't have to be public. It must not be static or final and return nothing. If a callback method throws an unchecked `RuntimeException`, Hibernate will abort the operation and mark the current transaction for rollback. If a callback method declares and throws a checked `Exception`, Hibernate will wrap and treat it as a `RuntimeException`.

We can only use each callback annotation once in an entity listener class; that is, only one method may be annotated `@PostPersist`. Table 13.2 summarizes all the available callback annotations.

Table 13.2 Lifecycle callback annotations

Annotation	Description
<code>@PostLoad</code>	Triggered after an entity instance is loaded into the persistence context, either by identifier lookup, through navigation and proxy/collection initialization, or with a query. Also called after refreshing an already-persistent instance.
<code>@PrePersist</code>	Called immediately when <code>persist()</code> is called on an entity instance. Also called for <code>merge()</code> when an entity is discovered as transient, after the transient state is copied into a persistent instance. Also called for associated entities if we enable <code>CascadeType.PERSIST</code> .
<code>@PostPersist</code>	Called after the database operation for making an entity instance persistent is executed and an identifier value is assigned. This may be when <code>persist()</code> or <code>merge()</code> is invoked, or later when the persistence context is flushed if the identifier generator is pre-insert (see section 5.2.5). Also called for associated entities if we enable <code>CascadeType.PERSIST</code> .
<code>@PreUpdate</code> , <code>@PostUpdate</code>	Executed before and after the persistence context is synchronized with the database; that is, before and after flushing. Triggered only when the state of the entity requires synchronization (for example, because it's considered dirty).
<code>@PreRemove</code> , <code>@PostRemove</code>	Triggered when <code>remove()</code> is called or the entity instance is removed by cascading, and after deletion of the record in the database when the persistence context is flushed.

An entity listener class must be enabled for any entity we'd like to intercept, such as this `Item`:

```
Path: Ch13/callback/src/main/java/com/manning/javapersistence/ch13
➡ /filtering/callback/Item.java
```

```
@Entity
@EntityListeners(
    PersistEntityListener.class
)
public class Item {
    // . . .
}
```

The `@EntityListeners` annotation accepts an array of listener classes if we have several interceptors. If several listeners define callback methods for the same event, Hibernate invokes the listeners in the declared order.

We don't have to write a separate entity listener class to intercept lifecycle events. We can, for example, implement the `logMessage()` method on the `User` entity class:

```
Path: Ch13/callback/src/main/java/com/manning/javapersistence/ch13
➡ /filtering/callback/User.java
```

```
@Entity
@Table(name = "USERS")
public class User {
    @PostPersist
    public void logMessage(){
        User currentUser = CurrentUser.INSTANCE.get();
        Log log = Log.INSTANCE;
        log.save(
            "Entity instance persisted by "
            + currentUser.getUsername()
            + ": "
            + this
        );
    }
    // . . .
}
```

Note that callback methods on an entity class don't have any arguments: the "current" entity involved in the state changes is `this`. Duplicate callbacks for the same event aren't allowed in a single class, but we can intercept the same event with callback methods in several listener classes or in a listener and an entity class.

We can also add callback methods on an entity superclass for the entire hierarchy. If, for a particular entity subclass, we want to disable the superclass's callbacks, we can annotate the subclass with `@ExcludeSuperclassListeners`. If we want to disable a default entity listener for a particular entity, we can mark it with the `@ExcludeDefaultListeners` annotation:

```
Path: Ch13/callback/src/main/java/com/manning/javapersistence/ch13
➡ /filtering/callback/User.java
```

```
@Entity
@Table(name = "USERS")
@ExcludeDefaultListeners
public class User {
    // . . .
}
```

JPA event listeners and callbacks provide a rudimentary framework for reacting to lifecycle events with our own procedures. Hibernate also has a more fine-grained and powerful alternative API: `org.hibernate.Interceptor`.

13.2.2 Implementing Hibernate interceptors

Let's assume that we want to write an audit log of data modifications in a separate database table. For example, we may want to record information about creation and update events for each `Item`. The audit log includes the user, the date and time of the event, what type of event occurred, and the identifier of the `Item` that was changed.

Audit logs are often handled using database triggers. On the other hand, it's sometimes better for the application to take responsibility, especially if portability between different databases is required.

We need several elements to implement audit logging. First, we have to mark the entity classes for which we want to enable audit logging. Next, we define what information to log, such as the user, date, time, and type of modification. Finally, we'll tie it all together with an `org.hibernate.Interceptor` that automatically creates the audit trail.

First we'll create a marker interface, `Auditable`:

```
Path: Ch13/interceptor/src/main/java/com/manning/javapersistence/ch13
➡ /filtering/interceptor/Auditable.java
```

```
public interface Auditable {
    Long getId();
}
```

This interface requires that a persistent entity class exposes its identifier with a getter method; we need this property to log the audit trail. Enabling audit logging for a particular persistent class is then trivial. We add it to the class declaration, such as for `Item`:

```
Path: /model/src/main/java/org/jpwh/model/filtering/interceptor/Item.java
```

```
@Entity
public class Item implements Auditable {
```

```

        // . . .
    }

```

Now we can create a new persistent entity class, `AuditLogRecord`, with the information we want to log in the audit database table:

```

Path: Ch13/interceptor/src/main/java/com/manning/javapersistence/ch13
➡ /filtering/interceptor/AuditLogRecord.java

```

```

@Entity
public class AuditLogRecord {
    @Id
    @GeneratedValue(generator = Constants.ID_GENERATOR)
    private Long id;
    @NotNull
    private String message;
    @NotNull
    private Long entityId;
    @NotNull
    private Class<? extends Auditable> entityClass;
    @NotNull
    private Long userId;
    @NotNull
    private LocalDateTime createdOn = LocalDateTime.now();
    // . . .
}

```

We want to store an instance of `AuditLogRecord` whenever Hibernate inserts or updates an `Item` in the database. A Hibernate interceptor can handle this automatically. Instead of implementing all methods in `org.hibernate.Interceptor`, we extend the `EmptyInterceptor` and override only the methods we need, as demonstrated in listing 13.2. We need to access the database to write the audit log, so the interceptor needs a Hibernate `Session`.

We'll also want to store the identifier of the currently logged-in user in each audit log record. The `inserts` and `updates` instance variables that we'll declare will be collections where this interceptor will hold its internal state.

Listing 13.2 Hibernate interceptor logging modification events

```

Path: Ch13/interceptor/src/test/java/com/manning/javapersistence/ch13
➡ /filtering/AuditLogInterceptor.java

```

```

public class AuditLogInterceptor extends EmptyInterceptor {
    private Session currentSession;
    private Long currentUserId;
    private Set<Auditable> inserts = new HashSet<>();
    private Set<Auditable> updates = new HashSet<>();
    public void setCurrentSession(Session session) {
        this.currentSession = session;
    }
    public void setCurrentUserId(Long currentUserId) {

```

```

        this.currentUserId =currentUserId;
    }
    public boolean onSave(Object entity, Serializable id,
        Object[] state, String[] propertyNames,
        Type[] types)
        throws CallbackException {
        if (entity instanceof Auditable aud) {
            inserts.add(aud);
        }
        return false;
    }
    public boolean onFlushDirty(Object entity, Serializable id,
        Object[] currentState,
        Object[] previousState,
        String[] propertyNames, Type[] types)
        throws CallbackException {
        if (entity instanceof Auditable aud) {
            updates.add(aud);
        }
        return false;
    }
    // . . .
}

```

Ⓐ This method is called when an entity instance is made persistent.

Ⓑ The `state` was not modified.

Ⓒ This method is called when an entity instance is detected as dirty during flushing of the persistence context.

Ⓓ The `currentState` was not modified.

The interceptor collects the modified `Auditable` instances in `inserts` and `updates`. Note that in `onSave()`, there may not be an identifier value assigned to the given entity instance. Hibernate guarantees to set entity identifiers during flushing, so the actual audit log trail is written in the `postFlush()` callback, which isn't shown in listing 13.2. This method is called after the flushing of the persistence context is complete.

Now we'll write the audit log records for all insertions and updates we collected earlier:

```

Path: Ch13/interceptor/src/test/java/com/manning/javapersistence/ch13
➡ /filtering/AuditLogInterceptor.java

public class AuditLogInterceptor extends EmptyInterceptor {
    // . . .
    public void postFlush(@SuppressWarnings("rawtypes") Iterator iterator)
        ➡ throws CallbackException {
        Session tempSession =
            currentSession.sessionWithOptions()
                .connection()
                .openSession();
        try {
            for (Auditable entity : inserts) {

```

```

        tempSession.persist(
            new AuditLogRecord("insert", entity, currentUserId)
        );
    }
    for (Auditable entity : updates) {
        tempSession.persist(
            new AuditLogRecord("update", entity, currentUserId)
        );
    }
    tempSession.flush();
} finally {
    tempSession.close();
    inserts.clear();
    updates.clear();
}
}
}

```

Ⓐ We can't access the original persistence context—the `Session` that is currently executing this interceptor. The `Session` is in a fragile state during interceptor calls. Hibernate lets us create a new `Session` that inherits some information from the original `Session` with the `sessionWithOptions()` method. The new temporary `Session` works with the same transaction and database connection as the original `Session`.

Ⓑ We store a new `AuditLogRecord` for each insertion and update using the temporary `Session`.

Ⓒ We flush and close the temporary `Session` independently of the original `Session`.

We're now ready to enable this interceptor:

```

Path: Ch13/interceptor/src/test/java/com/manning/javapersistence/ch13
➡ /filtering/AuditLogging.java

```

```

EntityManager em = emf.createEntityManager();
SessionFactory sessionFactory = emf.unwrap(SessionFactory.class);
Session session = sessionFactory.withOptions().
    interceptor(new AuditLogInterceptor()).openSession();

```

Enabling default interceptors

If we want to enable an interceptor by default for any `EntityManager`, we can set the `hibernate.ejb.interceptor` property in `persistence.xml` to a class that implements `org.hibernate.Interceptor`. Note that, unlike a session-scoped interceptor, Hibernate shares this default interceptor, so it must be thread-safe! The example `AuditLogInterceptor` is *not* thread-safe.

This `Session` now has an enabled `AuditLogInterceptor`, but the interceptor must also be configured with the current `Session` and logged-

in user identifier. This involves some typecasts to access the Hibernate API:

```
Path: Ch13/interceptor/src/test/java/com/manning/javapersistence/ch13
➡ /filtering/AuditLogging.java

AuditLogInterceptor interceptor =
    (AuditLogInterceptor) ((SessionImplementor) session).getInterceptor();
interceptor.setCurrentSession(session);
interceptor.setCurrentUserId(CURRENT_USER_ID);
```

The `Session` is now ready for use, and an audit trail will be written whenever we store or modify an `Item` instance with it.

Hibernate interceptors are flexible, and, unlike JPA event listeners and callback methods, we have access to much more contextual information when an event occurs. Having said that, Hibernate allows us to hook even deeper into its core with the extensible event system it's based on.

13.2.3 The core event system

The Hibernate core engine is based on a model of events and listeners. For example, if Hibernate needs to save an entity instance, it triggers an event. Whoever listens to this kind of event can catch it and handle saving the data. Hibernate, therefore, implements all of its core functionality as a set of default listeners, which can handle all Hibernate events.

Hibernate is open by design: we can write and enable our own listeners for Hibernate events. We can either replace the existing default listeners or extend them and execute a side effect or additional procedure. Replacing the event listeners is rare; doing so implies that our own listener implementation can take care of a piece of Hibernate's core functionality.

Essentially, all the methods of the `Session` interface (and its narrower cousin, the `EntityManager`) correlate to an event. The `find()` and `load()` methods trigger a `LoadEvent`, and by default this event is processed with the `DefaultLoadEventListener`.

A custom listener should implement the appropriate interface for the event it wants to process and/or extend one of the convenience base classes provided by Hibernate, or any of the default event listeners. Here's an example of a custom load event listener.

Listing 13.3 Custom load event listener

```
Path: Ch13/interceptor/src/test/java/com/manning/javapersistence/ch13
➡ /filtering/SecurityLoadListener.java

public class SecurityLoadListener extends DefaultLoadEventListener {
    public void onLoad(LoadEvent event, LoadType loadType)
        throws HibernateException {
        boolean authorized =
```

```

        MySecurity.isAuthorized(
            event.getEntityClassName(), event.getEntityId()
        );
        if (!authorized) {
            throw new MySecurityException("Unauthorized access");
        }
        super.onLoad(event, loadType);
    }
}

```

This listener performs custom authorization code. A listener should be considered effectively a singleton, meaning it's shared between persistence contexts and thus shouldn't save any transaction-related state as instance variables. For a list of all events and listener interfaces in native Hibernate, see the API Javadoc of the `org.hibernate.event` package.

We enable listeners for each core event in `persistence.xml`:

Path: `Ch13/interceptor/src/main/resources/META-INF/persistence.xml`

```

<properties>
    // . . .
    <property name="hibernate.ejb.event.load" value=
        "com.manning.javapersistence.ch13.filtering.SecurityLoadListener"/>
</properties>

```

The property name of the configuration settings always starts with `hibernate.ejb.event`, followed by the type of event we want to listen to. You can find a list of all event types in `org.hibernate.event.spi.EventType`. The value of the property can be a comma-separated list of listener class names; Hibernate will call each listener in the specified order.

We rarely have to extend the Hibernate core event system with our own functionality. Most of the time, an `org.hibernate.Interceptor` is flexible enough. However, it helps to have more options and to be able to replace any piece of the Hibernate core engine in a modular fashion.

The audit-logging implementation we demonstrated in the previous section was very simple. If we needed to log more information for auditing, such as the actually changed property values of an entity, we would consider Hibernate Envers.

13.3 Auditing and versioning with Hibernate Envers

Envers is a project of the Hibernate suite dedicated to audit logging and keeping multiple versions of data in the database. This is similar to version control systems you may already be familiar with, such as Subversion and Git.

With Envers enabled, a copy of the data is automatically stored in separate database tables when we add, modify, or delete data in the main tables of the application. Envers internally uses the Hibernate event SPI you saw in the previous section. Envers listens to Hibernate events, and when Hibernate stores changes in the database, Envers creates a copy of the data and logs a revision in its own tables.

Envers groups all data modifications in a unit of work—that is, in a transaction—as a change set with a revision number. We can write queries with the Envers API to retrieve historical data, given a revision number or timestamp; for example, “find all `Item` instances as they were last Friday.”

Once you enable Envers in your application, you will be able to easily work with it, as it is based on annotations. It will provide you with the option to keep multiple versions of data in the database with little effort on your part. The tradeoff is that it will create a lot of additional tables (but you will be able to control which tables you want to audit).

13.3.1 Enabling audit logging

Envers is available without further configuration as soon as we put its JAR file on the classpath (we’ll include it as a Maven dependency). We can enable audit logging selectively for an entity class with the `@org.hibernate.envers.Audited` annotation.

Listing 13.4 Enabling audit logging for the `Item` entity

```
Path: Ch13/envers/src/main/java/com/manning/javapersistence/ch13
➡ /filtering/envers/Item.java
```

```
@Entity
@org.hibernate.envers.Audited
public class Item {
    @NotNull
    private String name;
    @OneToMany(mappedBy = "item")
    @org.hibernate.envers.NotAudited
    private Set<Bid> bids = new HashSet<>();
    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "SELLER_ID", nullable = false)
    private User seller;
    // . . .
}
```

We’ve now enabled audit logging for `Item` instances and all properties of the entity. To disable audit logging for a particular property, we can annotate it with `@NotAudited`. In this case, Envers ignores the `bids` but audits the `seller`. We also have to enable auditing with `@Audited` on the `User` class.

Hibernate will now generate (or expect) additional database tables to hold historical data for each `Item` and `User`. Figure 13.1 shows the

schema for these tables.

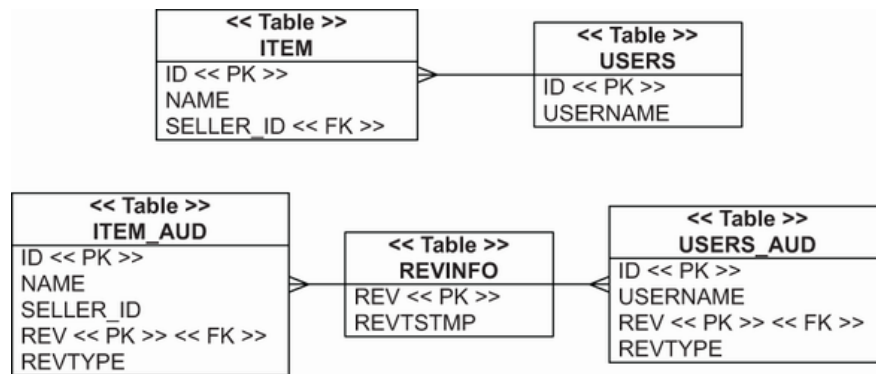


Figure 13.1 Audit logging tables for the `Item` and `User` entities

The `ITEM_AUD` and `USERS_AUD` tables are where the modification history of `Item` and `User` instances is stored. When we modify data and commit a transaction, Hibernate inserts a new revision number with a timestamp into the `REVINFO` table. Then, for each modified and audited entity instance involved in the change set, a copy of its data is stored in the audit tables. Foreign keys on revision number columns link the change set together. The `REVTYPE` column holds the type of change: whether the entity instance was inserted, updated, or deleted in the transaction. Envers never automatically removes any revision information or historical data; even after we `remove()` an `Item` instance, we still have its previous versions stored in `ITEM_AUD`.

Let's run through some transactions to see how this works.

13.3.2 Creating an audit trail

In the following code examples, we'll look at several transactions involving an `Item` and its `seller`, a `User`. We'll create and store an `Item` and `User`, then modify both, and then finally delete the `Item`.

You should already be familiar with this code. Envers automatically creates an audit trail when we work with the `EntityManager`:

```
Path: Ch13/envers/src/test/java/com/manning/javapersistence/ch13/filtering
➡ /Envers.java
```

```
EntityManager em = emf.createEntityManager();
em.getTransaction().begin();
User user = new User("johndoe");
em.persist(user);
Item item = new Item("Foo", user);
em.persist(item);
em.getTransaction().commit();
em.close();
```

```
Path: Ch13/envers/src/test/java/com/manning/javapersistence/ch13/filtering
➡ /Envers.java
```

```
EntityManager em = emf.createEntityManager();
em.getTransaction().begin();
```

```

Item item = em.find(Item.class, ITEM_ID);
item.setName("Bar");
item.getSeller().setUsername("doejohn");
em.getTransaction().commit();
em.close();

Path: Ch13/envers/src/test/java/com/manning/javapersistence/ch13/filtering
➡ /Envers.java

EntityManager em = emf.createEntityManager();
em.getTransaction().begin();
EntityManager em = JPA.createEntityManager();
Item item = em.find(Item.class, ITEM_ID);
em.remove(item);
em.getTransaction().commit();
em.close();

```

Envers transparently writes the audit trail for this sequence of transactions by logging three change sets. To access this historical data, we first have to obtain the number of the revision, representing the change set we'd like to access.

13.3.3 Finding revisions

With the Envers `AuditReader` API, we can find the revision number of each change set. The main Envers API is `AuditReader`. It can be accessed with an `EntityManager`. Given a timestamp, we can find the revision number of a change set made before or on that timestamp. If we don't have a timestamp, we can get all the revision numbers in which a particular audited entity instance was involved.

Listing 13.5 Obtaining the revision numbers of change sets

```

Path: Ch13/envers/src/test/java/com/manning/javapersistence/ch13/filtering
➡ /Envers.java

AuditReader auditReader = AuditReaderFactory.get(em);
Number revisionCreate =
    auditReader.getRevisionNumberForDate(TIMESTAMP_CREATE);
Number revisionUpdate =
    auditReader.getRevisionNumberForDate(TIMESTAMP_UPDATE);
Number revisionDelete =
    auditReader.getRevisionNumberForDate(TIMESTAMP_DELETE);
List<Number> itemRevisions =
    auditReader.getRevisions(Item.class, ITEM_ID);
assertEquals(3, itemRevisions.size());
for (Number itemRevision : itemRevisions) {
    Date itemRevisionTimestamp = auditReader.getRevisionDate(itemRevision);
    // . . .
}
List<Number> userRevisions =
    auditReader.getRevisions(User.class, USER_ID);
assertEquals(2, userRevisions.size());

```

Ⓐ Access the `AuditReader` Envers API.

- Ⓑ Find the revision number of a change set made before or on that timestamp.
- Ⓒ Without a timestamp, this operation finds all change sets where the given `Item` was created, modified, or deleted. In our example, we created, modified, and then deleted the `Item`. Hence, we have three revisions.
- Ⓓ If we have a revision number, we can get the timestamp when Envers logged the change set.
- Ⓔ We created and modified the `User`, so there are two revisions.

In listing 13.5, we assumed that either we know the (approximate) timestamp for a transaction or we have the identifier value of an entity so we can obtain its revisions. If we have neither, we may want to explore the audit log with queries. This is also useful if we have to show a list of all change sets in the user interface of the application.

The following code discovers all revisions of the `Item` entity class and loads each `Item` version and the audit log information for that change set:

```
Path: Ch13/envers/src/test/java/com/manning/javapersistence/ch13/filtering
➡ /Envers.java

AuditQuery query = auditReader.createQuery()                                Ⓐ
    .forRevisionsOfEntity(Item.class, false, false);
@SuppressWarnings("unchecked")
List<Object[]> result = query.getResultList();                               Ⓑ
for (Object[] tuple : result) {
    Item item = (Item) tuple[0];                                           Ⓒ
    DefaultRevisionEntity revision = (DefaultRevisionEntity)tuple[1];
    RevisionType revisionType = (RevisionType)tuple[2];
    if (revision.getId() == 1) {                                           Ⓓ
        assertEquals(RevisionType.ADD, revisionType);
        assertEquals("Foo", item.getName());
    } else if (revision.getId() == 2) {
        assertEquals(RevisionType.MOD, revisionType);
        assertEquals("Bar", item.getName());
    } else if (revision.getId() == 3) {
        assertEquals(RevisionType.DEL, revisionType);
        assertNull(item);
    }
}
```

- Ⓐ If we don't know the modification timestamps or revision numbers, we can write a query with `forRevisionsOfEntity()` to obtain all audit trail details of a particular entity.
- Ⓑ This query returns the audit trail details as a `List` of `Object[]`.
- Ⓒ Each result tuple contains the entity instance for a particular revision, the revision details (including revision number and timestamp), as well as the revision type.

① The revision type indicates why Envers created the revision—whether the entity instance was inserted, modified, or deleted in the database.

Revision numbers are sequentially incremented; a higher revision number is always a more recent version of an entity instance. We now have revision numbers for the three change sets in the audit trail, giving us access to historical data.

13.3.4 Accessing historical data

With a revision number, we can access different versions of the `Item` and its `seller`.

Listing 13.6 Loading historical versions of entity instances

```
Path: Ch13/envers/src/test/java/com/manning/javapersistence/ch13/filtering
➡ /Envers.java
```

```
Item item = auditReader.find(Item.class, ITEM_ID, revisionCreate); ①
assertEquals("Foo", item.getName());
assertEquals("johndoe", item.getSeller().getUsername());
Item modifiedItem = auditReader.find(Item.class,                      ②
    ITEM_ID, revisionUpdate);
assertEquals("Bar", modifiedItem.getName());
assertEquals("doejohn", modifiedItem.getSeller().getUsername());
Item deletedItem = auditReader.find(Item.class,                      ③
    ITEM_ID, revisionDelete);
assertNull(deletedItem);
User user = auditReader.find(User.class,                             ④
    USER_ID, revisionDelete);
assertEquals("doejohn", user.getUsername());
```

① The `find()` method returns an audited entity instance version, given a revision. This operation loads the `Item` as it was after creation.

② The `seller` of this change set is also retrieved automatically.

③ In this revision, the `Item` was deleted, so `find()` returns `null`.

④ The example didn't modify the `User` in this revision, so Envers returns its closest historical revision.

The `AuditReader#find()` operation retrieves only a single entity instance, like `EntityManager#find()`. But the returned entity instances are *not* in persistent state: the persistence context doesn't manage them. If we modify an older version of `Item`, Hibernate won't update the database. Consider the entity instances returned by the `AuditReader` API to be detached, or read-only.

`AuditReader` also has an API for the execution of arbitrary queries, similar to the native Hibernate `Criteria` API.

Listing 13.7 Querying historical entity instances

```
Path: Ch13/envers/src/test/java/com/manning/javapersistence/ch13/filtering
➡ /Envers.java
```

```
AuditQuery query = auditReader.createQuery()
    .forEntitiesAtRevision(Item.class, revisionUpdate);
query.add(AuditEntity.property("name").like("Ba", MatchMode.START));
query.add(AuditEntity.relatedId("seller").eq(USER_ID));
query.addOrder(AuditEntity.property("name").desc());
query.setFirstResult(0);
query.setMaxResults(10);
assertEquals(1, query.getResultList().size());
Item result = (Item)query.getResultList().get(0);
assertEquals("doejohn", result.getSeller().getUsername());
```

- Ⓐ This query returns `Item` instances restricted to a particular revision and change set.
- Ⓑ We can add further restrictions to the query; here the `Item#name` must start with “Ba”.
- Ⓒ Restrictions can include entity associations; for example, we’re looking for the revision of an `Item` sold by a particular `User`.
- Ⓓ We can order query results.
- Ⓔ We can paginate through large results.

Envers supports projection. The following query retrieves only the `Item#name` of a particular version:

```
Path: Ch13/envers/src/test/java/com/manning/javapersistence/ch13/filtering
➡ /Envers.java
```

```
AuditQuery query = auditReader.createQuery()
    .forEntitiesAtRevision(Item.class, revisionUpdate);
query.addProjection(AuditEntity.property("name"));
assertEquals(1, query.getResultList().size());
String result = (String)query.getSingleResult();
assertEquals("Bar", result);
```

Finally, we may want to roll back an entity instance to an older version. This can be accomplished with the `Session#replicate()` operation and overwriting an existing row. The following example loads the `User` instance from the first change set and then overwrites the current `User` in the database with the older version:

```
Path: Ch13/envers/src/test/java/com/manning/javapersistence/ch13/filtering
➡ /Envers.java
```

```
User user = auditReader.find(User.class, USER_ID, revisionCreate);
em.unwrap(Session.class)
    .replicate(user, ReplicationMode.OVERWRITE);
em.flush();
```

```
em.clear();
user = em.find(User.class, USER_ID);
assertEquals("johndoe", user.getUsername());
```

Envers will also track this change as an update in the audit log; it's just another new revision of the `User` instance.

Temporal data is a complex subject, and we encourage you to read the Envers reference documentation for more information. Adding details to the audit log, such as the user who made a change, isn't difficult. The documentation also shows how to configure different tracking strategies and customize the database schema used by Envers.

Next, imagine that you don't want to see all the data in the database. For example, the currently logged-in application user may not have the rights to see everything. Usually, we add a condition to the queries and restrict the results dynamically. But this becomes difficult if we have to handle a concern such as security, because we'd have to customize most of the queries in the application. We can centralize and isolate these restrictions with Hibernate's dynamic data filters.

13.4 Dynamic data filters

The first use case for dynamic data filtering relates to data security. A `User` in `CaveatEmptor` may have a `ranking` property, which is a simple integer:

```
Path: /model/src/main/java/org/jpwh/model/filtering/dynamic/User.java
```

```
@Entity
@Table(name = "USERS")
public class User {
    @NotNull
    private int ranking = 0;
    // . . .
}
```

Now assume that users can only bid on items that other users offer with an equal or lower ranking. In business terms, we may have several groups of users that are defined by an arbitrary ranking (a number), and users can trade only with people who have the same or a lower ranking.

To implement this requirement, we'd have to customize all queries that load `Item` instances from the database. We'd check whether the `Item#seller` we want to load has an equal or lower ranking than the currently logged-in user. Hibernate can do this work for us with a dynamic filter.

13.4.1 Defining dynamic filters

First we'll define our filter with a name and the dynamic runtime parameters it accepts. We can place the Hibernate annotation for this defini-

tion on any entity class of our domain model or in a package-info.java metadata file:

```
Path: Ch13/dynamic/src/main/java/com/manning/javapersistence/ch13/filtering
➡ /dynamic/package-info.java

@org.hibernate.annotations.FilterDef(
    name = "limitByUserRanking",
    parameters = {
        @org.hibernate.annotations.ParamDef(
            name = "currentUserRanking", type = "int"
        )
    }
)
```

This example names this filter `limitByUserRanking`; note that filter names must be unique in a persistence unit. It accepts one runtime argument of type `int`. If we have several filter definitions, we declare them within `@org.hibernate.annotations.FilterDefs`.

The filter is inactive now; nothing indicates that it's supposed to apply to `Item` instances. We must apply and implement the filter on the classes or collections we want to filter.

13.4.2 Applying a dynamic filter

We want to apply the defined filter on the `Item` class so that no items are visible if the logged-in user doesn't have the necessary rank:

```
Path: Ch13/dynamic/src/main/java/com/manning/javapersistence/ch13/filtering
➡ /dynamic/Item.java

@Entity
@org.hibernate.annotations.Filter(
    name = "limitByUserRanking",
    condition = ""
        :currentUserRanking >= (
            select u.RANKING from USERS u
            where u.ID = SELLER_ID
        ) ""
)
public class Item {
    // . . .
}
```

The `condition` in the preceding code is an SQL expression that's passed through directly to the database system, so we can use any SQL operator or function. It must evaluate to `true` if a record should pass the filter. In this example, we are using a subquery to obtain the `ranking` of the seller of the item. Unqualified columns, such as `SELLER_ID`, refer to the table mapped to the entity class. If the currently logged-in user's ranking isn't greater than or equal to the ranking returned by the subquery, the

`Item` instance is filtered out. We can apply several filters by grouping them in an `@org.hibernate.annotations.Filters`.

A defined and applied filter, if enabled for a particular unit of work, filters out any `Item` instance that doesn't pass the condition. Let's enable it.

13.4.3 Enabling a dynamic filter

We've defined a data filter and applied it to a persistent entity class. It's still not filtering anything—it must be enabled and parameterized in the application for a particular unit of work, with the `Session` API:

```
Path: Ch13/dynamic/src/test/java/com/manning/javapersistence/ch13/filtering
➡ /DynamicFilter.java
```

```
org.hibernate.Filter filter = em.unwrap(Session.class)
    .enableFilter("limitByUserRanking");
filter.setParameter("currentUserRanking", 0);
```

We enable the filter by name, and the method returns a `Filter` on which we set the runtime arguments dynamically. We must set the parameters we've defined; here it's set to ranking 0. This example then filters out `Items` sold by a `User` with a higher ranking in this `Session`.

Other useful methods of the `Filter` are `getFilterDefinition()`, which allows us to iterate through the parameter names and types, and `validate()`, which throws a `HibernateException` if we forget to set a parameter. We can also set a list of arguments with `setParameterList()`; this is mostly useful if our SQL restriction contains an expression with a quantifier operator (the `IN` operator, for example).

Now every JPQL or criteria query that we execute on the filtered persistence context restricts the returned `Item` instances:

```
Path: Ch13/dynamic/src/test/java/com/manning/javapersistence/ch13/filtering
➡ /DynamicFilter.java
```

```
List<Item> items = em.createQuery("select i from Item i", Item.class).getRe
// select * from ITEM where 0 >=
// (select u.RANKING from USERS u where u.ID = SELLER_ID)
```

```
Path: Ch13/dynamic/src/test/java/com/manning/javapersistence/ch13/filtering
➡ /DynamicFilter.java
```

```
CriteriaBuilder cb = em.getCriteriaBuilder();
CriteriaQuery<Item> criteria = cb.createQuery(Item.class);
criteria.select(criteria.from(Item.class));
List<Item> items = em.createQuery(criteria).getResultList();
// select * from ITEM where 0 >=
// (select u.RANKING from USERS u where u.ID = SELLER_ID)
```

Note how Hibernate dynamically appends the SQL restriction conditions to the statement generated.

When you first experiment with dynamic filters, you'll most likely run into a problem with retrieval by identifier. You might expect that `em.find(Item.class, ITEM_ID)` will be filtered as well. This is not the case, though: Hibernate doesn't apply filters to retrieval by identifier operations. One of the reasons is that data-filter conditions are SQL fragments, and lookup by identifier may be resolved completely in memory in the first-level persistence context cache. Similar reasoning applies to filtering of many-to-one or one-to-one associations. If a many-to-one association was filtered (for example, by returning `null` if you called `anItem.getSeller()`), the multiplicity of the association would change! You won't know if the item has a seller or if you aren't allowed to see it.

But you can dynamically filter collection access.

13.4.4 Filtering collection access

Until now, calling `someCategory.getItems()` has returned all `Item` instances that are referenced by that `Category`. This can be restricted with a filter applied to a collection:

```
Path: Ch13/dynamic/src/main/java/com/manning/javapersistence/ch13/filtering
➡ /dynamic/Category.java
```

```
@Entity
public class Category {
    @OneToMany(mappedBy = "category")
    @org.hibernate.annotations.Filter(
        name = "limitByUserRanking",
        condition = ""
            :currentUserRanking >= (
                select u.RANKING from USERS u
                where u.ID = SELLER_ID
            )""
    )
    private Set<Item> items = new HashSet<>();
    // . . .
}
```

If we now enable the filter in a `Session`, all iteration through a collection of `Category# items` is filtered:

```
Path: Ch13/dynamic/src/test/java/com/manning/javapersistence/ch13/filtering
➡ /DynamicFilter.java
```

```
filter.setParameter("currentUserRanking", 0);
Category category = em.find(Category.class, CATEGORY_ID);
assertEquals(1, category.getItems().size());
```

If the current user's ranking is 0, only one `Item` is loaded when we access the collection. With a ranking of 100, we can see more data:

```
Path: Ch13/dynamic/src/test/java/com/manning/javapersistence/ch13/filtering
➡ /DynamicFilter.java
```

```

filter.setParameter("currentUserRanking", 100);
category = em.find(Category.class, CATEGORY_ID);
assertEquals(2, category.getItems().size());

```

You probably noticed that the SQL condition for both filter applications is the same. If the SQL restriction is the same for all filter applications, we can set it as the default condition when we define the filter, so we don't have to repeat it:

```

Path: Ch13/dynamic/src/main/java/com/manning/javapersistence/ch13/filtering
➡ /dynamic/package-info.java

```

```

@org.hibernate.annotations.FilterDef(
    name = "limitByUserRankingDefault",
    defaultCondition= ""
        :currentUserRanking >= (
            select u.RANKING from USERS u
            where u.ID = SELLER_ID
        ) """,
    parameters = {
        @org.hibernate.annotations.ParamDef(
            name = "currentUserRanking", type = "int"
        )
    }
)

```

There are many other excellent use cases for dynamic data filters. We've seen a restriction of data access given an arbitrary security-related condition. This can be the user ranking, a particular group the user must belong to, or a role the user has been assigned. Data might be stored with a regional code (for example, all business contacts of a sales team). Or perhaps each salesperson works only on data that covers their region.

Summary

- Cascading state transitions are predefined reactions to lifecycle events in the persistence engine.
- Cascading provides options: the transitive detachment and merging, the cascading refresh and replication.
- You can implement event listeners and interceptors to add custom logic when Hibernate loads and stores data.
- You can use Hibernate Envers for audit logging and keeping multiple versions of data in the database (like a version control system).
- You can query Envers to retrieve historical data.
- You can use dynamic data filters so that Hibernate can automatically append arbitrary SQL restrictions to queries it generates.
- You can define dynamic filters, apply and enable the filters, and filter collection access.

